

PRESENTATION GENERALE DU COURS

1. Importance du Cours

Dans un monde numérique en constante évolution, Linux s'est imposé comme le système d'exploitation de référence dans les environnements professionnels, scientifiques et industriels. Aujourd'hui, plus de 90 % des serveurs qui font fonctionner Internet, la quasi-totalité des supercalculateurs mondiaux, les infrastructures Cloud de Google, Amazon et Microsoft, ainsi que la plupart des systèmes embarqués et des appareils mobiles Android reposent sur des fondations Linux.

Pour un étudiant en Bachelor Informatique, la maîtrise de Linux n'est pas un simple avantage, c'est une compétence fondamentale et incontournable. Que vous vous orientiez vers le développement logiciel, la cybersécurité, l'administration systèmes, le DevOps, la data science ou l'intelligence artificielle, vous serez amené à travailler quotidiennement dans des environnements Linux.

Ce cours vous offre une immersion complète et progressive dans cet écosystème. Il vous permettra non seulement de comprendre les mécanismes internes d'un système d'exploitation UNIX-like, mais aussi de développer une aisance opérationnelle qui vous distinguera sur le marché du travail.

Pourquoi Linux est indispensable à votre formation ?

- Linux alimente plus de 90 % des serveurs et infrastructures cloud mondiales
- Compétence exigée dans les métiers du DevOps, de la cybersécurité et du développement backend
- Fondation des environnements de Data Science, Machine Learning et HPC
- Système de prédilection pour le développement open source et les outils professionnels
- Maîtrise de la ligne de commande : facteur différenciant sur le marché de l'emploi
- Base essentielle pour comprendre les concepts d'administration systèmes et réseaux

2. Objectif Général

Ce cours a pour objectif général de former les étudiants de Bachelor 1 aux fondamentaux du système Linux, en leur fournissant les connaissances théoriques et les compétences pratiques nécessaires pour naviguer, administrer et automatiser un environnement Linux de manière autonome et efficace.

À l'issue de ce cours, l'étudiant sera capable d'installer et configurer un système Linux, de gérer ses ressources (fichiers, processus, utilisateurs, services), et d'écrire des scripts d'automatisation. Il disposera ainsi d'une base solide pour aborder les cours avancés en administration systèmes, réseaux, sécurité informatique et développement.

3. Objectifs Spécifiques

À l'issue de chacun des modules, l'étudiant sera capable d'atteindre les objectifs spécifiques suivants :

Chapitre 1 : Installation et Premiers Pas

- Installer et configurer une distribution Ubuntu dans un environnement virtualisé ou natif
- Comprendre l'architecture générale d'un système Linux et ses composants fondamentaux
- Utiliser le terminal Linux et exécuter des commandes de base en ligne de commande

Chapitre 2 : Système de Fichiers et Navigation

- Décrire et naviguer dans l'arborescence FHS (Filesystem Hierarchy Standard)
- Utiliser les commandes de navigation : ls, cd, pwd, find, locate
- Distinguer les chemins absolus et relatifs et les utiliser correctement

Chapitre 3 : Gestion des Fichiers et Permissions

- Créer, copier, déplacer, renommer et supprimer des fichiers et répertoires
- Comprendre et modifier les permissions Unix (rwx) avec chmod, chown et chgrp
- Maîtriser la notion de propriété et appliquer des politiques d'accès adaptées

Chapitre 4 : Les Processus et la Mémoire

- Lister, surveiller et gérer les processus actifs du système
- Utiliser les signaux Unix pour contrôler l'exécution des processus
- Analyser l'utilisation des ressources mémoire et CPU avec top, htop, free

Chapitre 5 : Gestion des Utilisateurs et Groupes

- Créer et gérer des comptes utilisateurs et des groupes sous Linux

- Configurer et utiliser les privilèges sudo de manière sécurisée
- Mettre en place une politique de sécurité des accès adaptée à un environnement multi-utilisateur

Chapitre 6 : Gestion des Paquets et Services

- Installer, mettre à jour et supprimer des logiciels via le gestionnaire APT
- Configurer et contrôler des services système avec systemd (start, stop, enable, status)
- Configurer des paramètres réseau de base sous Linux

Chapitre 7 : Introduction au Scripting Shell

- Écrire des scripts Bash fonctionnels utilisant variables, conditions et boucles
- Automatiser des tâches répétitives et les planifier avec cron
- Appliquer les bonnes pratiques de développement de scripts en environnement Linux

Chapitre 8 : Projet Final Intégré

- Concevoir et déployer une solution d'administration Linux intégrant les acquis des modules précédents
- Documenter et présenter une architecture système répondant à un cahier des charges défini
- Démontrer une autonomie opérationnelle dans la résolution de problèmes systèmes réels

4. Structure du Cours

Le cours est structuré en 8 modules progressifs, chacun d'une durée de 3 heures, pour un volume horaire total de 24 heures. La progression pédagogique est conçue selon une logique d'escalade des compétences : chaque module s'appuie sur les acquis des modules précédents et prépare aux notions suivantes.

MODULE	TITRE	OBJECTIF PRINCIPAL
CHAPITRE 1	Installation et Premiers Pas sous Linux	Découverte de l'environnement Linux, installation Ubuntu

MODULE	TITRE	OBJECTIF PRINCIPAL
CHAPITRE 2	Système de Fichiers et Navigation	Arborescence FHS, commandes de navigation, chemins
CHAPITRE 3	Gestion des Fichiers et Permissions	Manipulation fichiers, droits d'accès, propriété
CHAPITRE 4	Les Processus et la Mémoire	Gestion processus, signals, surveillance système
CHAPITRE 5	Gestion des Utilisateurs et Groupes	Création comptes, sudo, configuration sécurité
CHAPITRE 6	Gestion des Paquets et Services	APT, systemd, configuration services réseau
CHAPITRE 7	Introduction au Scripting Shell	Scripts Bash, automatisation, cron
CHAPITRE 8	Projet Final Intégré	Mise en situation réelle d'administration

5. Compétences Acquisées à l'Issue du Cours

À l'issue du cours, l'étudiant aura développé un ensemble de compétences structurées en trois domaines complémentaires : les compétences techniques opérationnelles, les compétences méthodologiques et transversales, et les compétences professionnelles.

5.1 Compétences Techniques Opérationnelles

Ce que vous saurez faire techniquement

- Installer, configurer et maintenir un système Linux Ubuntu (natif ou virtualisé)
- Naviguer et administrer le système de fichiers avec aisance depuis le terminal
- Gérer les permissions, la propriété et la sécurité des fichiers et répertoires

- Surveiller et contrôler les processus, la mémoire et les ressources système
- Créer et administrer des comptes utilisateurs et groupes avec gestion des droits
- Installer et gérer des paquets logiciels et des services système avec APT et systemd
- Écrire des scripts Bash pour automatiser des tâches d'administration
- Planifier des tâches automatiques avec cron et crontab

5.2 Compétences Méthodologiques et Transversales

Au-delà des compétences techniques, ce cours développe chez l'étudiant une méthode de travail rigoureuse et une autonomie dans la résolution de problèmes systèmes :

- Lecture et interprétation de la documentation technique (man pages, wiki, forums)
- Diagnostic et résolution méthodique des erreurs système (debugging)
- Rédaction de documentation technique claire et structurée
- Capacité à travailler en ligne de commande sans interface graphique
- Adoption des bonnes pratiques de sécurité dans la gestion d'un système
- Pensée systémique : comprendre les interdépendances entre composants du système

5.3 Compétences Professionnelles

Ces compétences vous permettront d'aborder sereinement les environnements professionnels et les cours avancés de votre cursus :

- Préparer et réussir les certifications Linux professionnelles (LPIC-1, CompTIA Linux+)
- Intégrer une équipe DevOps ou d'administration systèmes avec des bases solides
- Travailler efficacement dans des environnements de développement sous Linux ou macOS
- Appréhender les technologies Cloud (AWS, GCP, Azure) qui reposent sur des VMs Linux
- Contribuer à des projets open source hébergés sur des dépôts Linux

CHAPITRE I : INSTALLATION ET PREMIERS PAS SOUS LINUX

1.1 Introduction aux Systèmes d'Exploitation

1.1.1 Qu'est-ce qu'un Système d'Exploitation ?

Un système d'exploitation (Operating System en anglais) est le logiciel fondamental qui fait le lien entre le matériel informatique (processeur, mémoire, disque dur, écran, clavier) et les applications que vous utilisez quotidiennement. Sans système d'exploitation, votre ordinateur ne serait qu'un assemblage de composants électroniques incapables de communiquer entre eux ou d'exécuter des programmes. Le système d'exploitation gère les ressources matérielles, organise le stockage des fichiers, contrôle l'accès à la mémoire, et permet l'exécution simultanée de plusieurs programmes.

Imaginez votre ordinateur comme une grande entreprise. Le matériel (processeur, mémoire, disques) représente les employés et les bureaux. Les applications (navigateur web, traitement de texte, jeux) sont les projets à réaliser. Le système d'exploitation est le directeur général qui coordonne tout : il décide quel employé (processeur) travaille sur quel projet (application), où ranger les documents (fichiers), comment distribuer les ressources (mémoire), et s'assure que tout le monde peut travailler efficacement sans se marcher sur les pieds. Sous Linux, ce directeur est particulièrement transparent et vous permet de voir exactement comment il organise le travail.

1.1.2 L'Histoire de Linux et sa Place dans le Monde Informatique

Linux est né en 1991 grâce à Linus Torvalds, un étudiant finlandais de 21 ans qui voulait créer un système d'exploitation libre et gratuit pour son ordinateur personnel. Frustré par les limitations des systèmes existants, il a écrit les premières lignes de code du noyau Linux et les a partagées sur Internet. Sa vision était révolutionnaire : créer un système ouvert où n'importe qui pourrait voir le code source, le modifier, et l'améliorer. Contrairement aux systèmes propriétaires comme Windows ou macOS dont le code est secret, Linux adopte une philosophie de transparence totale. Ce qui a commencé comme un projet étudiant est devenu l'un des systèmes d'exploitation les plus utilisés au monde.

Aujourd'hui, Linux est omniprésent dans notre vie quotidienne, même si vous ne le voyez pas. Quand vous utilisez Google, Facebook, ou Netflix, vos requêtes sont traitées par des serveurs Linux. Votre téléphone Android ? C'est Linux. Les supercalculateurs qui font des prévisions météorologiques ou qui étudient le génome humain ? 96% d'entre eux tournent

sous Linux. Les routeurs qui font fonctionner Internet ? Linux. Les objets connectés dans votre maison, les voitures modernes, les systèmes embarqués dans les avions : Linux est partout. Au Cameroun et en Afrique, l'adoption de Linux dans l'éducation et l'administration publique progresse rapidement car il permet de réduire les coûts de licence tout en offrant une sécurité et une stabilité exceptionnelles. Pour un informaticien professionnel, maîtriser Linux n'est pas optionnel, c'est fondamental.

1.1.3 Les Distributions Linux : Ubuntu, Debian, CentOS et Autres

Une distribution Linux (souvent appelée « distro ») est un système d'exploitation complet construit autour du noyau Linux. Le noyau seul ne suffit pas : il faut y ajouter des milliers de logiciels (navigateur web, éditeur de texte, gestionnaire de fichiers), un installateur, des outils de configuration, et une interface graphique. Chaque distribution fait des choix différents sur ces éléments. Ubuntu, que nous utiliserons dans ce cours, est conçue pour être facile à utiliser et convient parfaitement aux débutants. Elle est développée par la société Canonical et bénéficie d'une énorme communauté d'utilisateurs. Debian, sa distribution mère, est réputée pour sa stabilité et est utilisée sur les serveurs critiques. CentOS et Red Hat Enterprise Linux dominent le monde professionnel et sont le standard dans les grandes entreprises.

Le choix d'une distribution dépend de vos besoins. Pour l'apprentissage et le desktop, Ubuntu est idéal grâce à son interface graphique moderne GNOME, sa documentation abondante en français, et sa facilité d'installation. Pour les serveurs web et les infrastructures cloud au monde, on utilise souvent Ubuntu Server ou Debian car ils sont gratuits, stables, et consomment peu de ressources. Dans ce cours de Bachelor I, nous nous concentrerons exclusivement sur Ubuntu 22.04 LTS (Long Term Support), une version soutenue pendant 5 ans par Canonical avec des mises à jour de sécurité régulières. Maîtriser Ubuntu vous donnera les bases nécessaires pour comprendre toutes les autres distributions, car elles partagent 90% de leurs commandes et concepts fondamentaux.

Distribution	Public cible	Caractéristique	Usage typique
Ubuntu	Grand public, débutants	Interface moderne, facile	Cours, desktop, dev web

Distribution	Public cible	Caractéristique	Usage typique
Debian	Serveurs, stabilité maximale	Très stable, conservateur	Serveurs production
CentOS / AlmaLinux	Entreprises, serveurs	Compatible Red Hat, gratuit	Data centers, clouds
Fedora	Développeurs, technologies récentes	Innovant, cutting-edge	Stations de travail
Arch Linux	Utilisateurs avancés	Minimaliste, personnalisable	Experts Linux
KALI Linux	Sécurité informatique	Outils pentest intégrés	Audit sécurité

1.2 Installation d'Ubuntu dans VirtualBox

1.2.1 Pourquoi Utiliser VirtualBox pour Apprendre Linux ?

VirtualBox est un logiciel de virtualisation gratuit développé par Oracle qui permet de faire fonctionner un système d'exploitation complet (appelé « machine virtuelle » ou VM) à l'intérieur de votre système actuel (appelé « hôte »). C'est comme avoir un ordinateur dans votre ordinateur. Pour apprendre Linux, c'est la solution idéale car elle est totalement sûre : quoi que vous fassiez dans la machine virtuelle (supprimer des fichiers système, casser la configuration, installer des logiciels suspects), votre système Windows principal reste intact et protégé. Vous pouvez expérimenter librement, faire des erreurs, tout casser, puis simplement restaurer un snapshot (instantané) pour revenir à un état antérieur en quelques secondes.

Pour un étudiant débutant, VirtualBox élimine la peur de l'erreur qui paralyse l'apprentissage. Vous n'avez pas besoin d'un deuxième ordinateur, ni de créer une partition risquée sur votre disque dur principal. Dans une salle de TP avec 60 étudiants, chacun peut avoir sa propre installation Ubuntu isolée, faire ses manipulations sans crainte, et même emporter sa machine virtuelle sur une clé USB pour continuer à pratiquer chez soi. De plus, VirtualBox permet de configurer facilement le réseau, de partager des fichiers entre l'hôte et

la VM via des dossiers partagés, et de prendre des snapshots avant chaque manipulation risquée. C'est l'environnement parfait pour transformer des étudiants ayant peur de la ligne de commande en futurs administrateurs système confiants.

1.2.2 Téléchargement et Installation de VirtualBox

La première étape consiste à télécharger VirtualBox depuis le site officiel www.virtualbox.org. Rendez-vous dans la section Downloads et choisissez la version correspondant à votre système d'exploitation hôte (Windows, macOS, ou Linux). Le téléchargement pèse environ 120 Mo et prend quelques minutes selon votre connexion Internet. Une fois le fichier téléchargé, lancez l'installateur en double-cliquant dessus. L'assistant d'installation vous guidera à travers plusieurs écrans : acceptez les paramètres par défaut en cliquant sur Next à chaque étape. Quand Windows vous demande d'autoriser l'installation de pilotes réseau pour VirtualBox, cliquez sur Oui ou Installer.

L'installation complète prend 2 à 3 minutes. À la fin, cochez la case pour démarrer VirtualBox automatiquement et cliquez sur Finish. VirtualBox s'ouvre : vous voyez une fenêtre avec une barre d'outils en haut et une liste vide au centre (puisque vous n'avez pas encore créé de machine virtuelle). L'interface est intuitive avec des icônes claires : Nouvelle (pour créer une VM), Démarrer (pour lancer une VM), Paramètres (pour configurer une VM).

1.2.3 Téléchargement de l'Image ISO d'Ubuntu

Une image ISO est un fichier qui contient l'intégralité du système d'exploitation sous forme d'une copie exacte d'un DVD d'installation. Pour Ubuntu 22.04 LTS, rendez-vous sur le site officiel ubuntu.com/download/desktop. Cliquez sur le bouton Download Ubuntu 22.04 LTS. Le fichier pèse environ 3,6 Go et se nomme `ubuntu-22.04.X-desktop-amd64.iso` (où X est le numéro de la version mineure). Le téléchargement peut prendre 20 à 60 minutes selon votre débit Internet. Si vous formez 60 étudiants et que la connexion du campus est limitée, il est plus judicieux de télécharger l'ISO une seule fois, de la placer sur un serveur local ou des clés USB, et de la distribuer aux étudiants. Cela évite de saturer la bande passante avec 60 téléchargements simultanés de 3,6 Go chacun.

Pendant que l'ISO se télécharge, expliquez aux étudiants ce qu'ils vont installer : Ubuntu 22.04 LTS est une version majeure publiée en avril 2022, supportée jusqu'en avril 2027 avec des mises à jour de sécurité régulières. Le terme LTS (Long Term Support) signifie que Canonical s'engage à maintenir cette version pendant 5 ans, ce qui en fait un choix sûr pour l'apprentissage et la production. L'interface par défaut est GNOME 42, un

environnement graphique moderne et élégant avec des fenêtres, des icônes, et un dock latéral similaire à macOS. Une fois le téléchargement terminé, vérifiez la taille du fichier (environ 3,6 Go) et son intégrité si possible en comparant le hash SHA256 fourni sur le site Ubuntu. Stockez ce fichier ISO dans un endroit facilement accessible comme Téléchargements ou le Bureau, car vous en aurez besoin dans l'étape suivante pour créer votre machine virtuelle.

1.2.4 Création de la Machine Virtuelle Ubuntu

Lancez VirtualBox. Dans la barre de menus, cliquez sur Machine > Nouvelle (ou utilisez le bouton bleu Nouvelle dans la barre d'outils). Une fenêtre assistant s'ouvre. Dans le champ Nom, saisissez un nom explicite comme Ubuntu-22.04-TP ou Linux-Bachelor1. VirtualBox détecte automatiquement que vous créez une machine Linux Ubuntu 64-bit en analysant le nom. Si ce n'est pas le cas, sélectionnez manuellement Type: Linux et Version: Ubuntu (64-bit) dans les menus déroulants. Cliquez sur Suivant. L'écran suivant vous demande combien de mémoire RAM allouer à la machine virtuelle. Pour Ubuntu avec interface graphique, le minimum recommandé est 2048 Mo (2 Go). Si votre ordinateur a 8 Go de RAM ou plus, vous pouvez allouer 4096 Mo (4 Go) pour une expérience plus fluide. Attention : ne dépassez pas la zone verte de la jauge, sinon votre système hôte manquera de mémoire et ralentira.

Cliquez sur Suivant. VirtualBox vous demande de créer un disque dur virtuel. Laissez l'option Créer un disque dur virtuel maintenant sélectionnée et cliquez sur Créer. Choisissez le type VDI (VirtualBox Disk Image) et cliquez sur Suivant. Pour la taille du disque, sélectionnez Dynamiquement alloué (le fichier disque ne consommera que l'espace réellement utilisé par la VM) et cliquez sur Suivant. Définissez la taille maximale à 25 Go minimum (30 Go recommandé pour installer confortablement des logiciels supplémentaires). Cliquez sur Créer. Votre machine virtuelle apparaît maintenant dans la liste de gauche de VirtualBox, mais elle est éteinte (icône grisée). Avant de la démarrer, vous devez insérer le DVD d'installation virtuel (l'image ISO d'Ubuntu). Sélectionnez votre VM dans la liste, puis cliquez sur Configuration. Allez dans l'onglet Stockage, cliquez sur l'icône CD vide sous Contrôleur IDE, puis sur l'icône de disque bleu à droite et choisissez Choisissez un fichier de disque. Naviguez jusqu'à votre fichier ubuntu-22.04.X-desktop-amd64.iso, sélectionnez-le et cliquez sur Ouvrir. Cliquez sur OK pour fermer la fenêtre de configuration. Votre VM est prête à démarrer.

- **Points d'attention pour l'installation en classe**

- ✓ Mémoire RAM : Vérifier que chaque PC étudiant a au moins 4 Go de RAM totale
- ✓ Virtualisation matérielle : Activer VT-x/AMD-V dans le BIOS si désactivé
- ✓ Espace disque : S'assurer d'au moins 40 Go libres sur le disque hôte
- ✓ Réseau : Configurer le mode NAT pour que la VM accède à Internet
- ✓ Sauvegarde : Prendre un snapshot après l'installation réussie

1.2.5 Installation d'Ubuntu Pas à Pas

Sélectionnez votre machine virtuelle dans VirtualBox et cliquez sur le bouton vert Démarrer. Une nouvelle fenêtre s'ouvre : c'est l'écran de votre machine virtuelle. Ubuntu démarre depuis l'ISO et affiche son menu d'accueil GRUB avec plusieurs options. Sélectionnez Try or Install Ubuntu (option par défaut) et appuyez sur Entrée. Après quelques secondes de chargement avec le logo Ubuntu et des points qui clignent, vous arrivez sur le bureau en mode Live (essai). Ubuntu fonctionne depuis la RAM sans rien installer sur le disque. Vous pouvez tester le système, naviguer sur Internet, ouvrir des applications. Pour installer définitivement Ubuntu sur le disque virtuel, double-cliquez sur l'icône Install Ubuntu 22.04 LTS sur le bureau. L'installateur graphique se lance.

Premier écran : sélectionnez votre langue (Français si vous préférez, mais je recommande English pour ce cours afin de vous habituer aux termes techniques en anglais, universellement utilisés en informatique). Cliquez Continue. Deuxième écran : disposition du clavier. Sélectionnez French (pour les claviers AZERTY camerounais) ou English US si vous avez un clavier QWERTY. Testez en tapant dans la zone de texte pour vérifier. Cliquez Continue. Troisième écran : mises à jour et logiciels. Choisissez Normal installation (inclut navigateur, bureautique, médias) et cochez Download updates while installing Ubuntu et Install third-party software. Cliquez Continue. Quatrième écran : type d'installation. Choisissez Erase disk and install Ubuntu (ne vous inquiétez pas, cela efface uniquement le disque virtuel vide de la VM, pas votre disque réel). Cliquez Install Now puis Continue pour confirmer. L'installateur écrit alors les partitions sur le disque virtuel.

Cinquième écran : fuseau horaire. Sélectionnez Africa/Douala pour le Cameroun. Cliquez Continue. Sixième écran : création de votre compte utilisateur. Remplissez votre nom (ex : Etudiant KIIA), un nom d'utilisateur pour vous connecter (ex : etudiant, en minuscules

sans espaces), un mot de passe sécurisé (minimum 8 caractères avec majuscules, chiffres, et symboles(notez-le bien !)), et confirmez le mot de passe. Cochez Require my password to log in pour la sécurité. Cliquez Continue. L'installation commence : une barre de progression indique l'avancement. Cette étape dure 10 à 20 minutes selon la vitesse du disque. L'installateur copie les fichiers, installe les logiciels, configure le système. Pendant ce temps, un diaporama vous présente les fonctionnalités d'Ubuntu. Quand la barre atteint 100%, un message Installation Complete apparaît. Cliquez sur Restart Now. La VM va redémarrer. Un message Please remove the installation medium, then press ENTER s'affiche. VirtualBox éjecte automatiquement l'ISO du lecteur CD virtuel, appuyez donc simplement sur Entrée.

1.2.6 Premier Démarrage et Configuration Initiale

Ubuntu redémarre et affiche l'écran de connexion avec votre nom d'utilisateur. Cliquez dessus, saisissez votre mot de passe, et appuyez sur Entrée. Vous voilà sur le bureau GNOME d'Ubuntu ! Un assistant de bienvenue se lance automatiquement. Parcourez les écrans : sautez la connexion aux comptes en ligne (Skip), désactivez les rapports d'erreur automatiques si vous le souhaitez (Next), et cliquez sur Done. Le bureau GNOME s'affiche : à gauche, un dock vertical avec des icônes d'applications (Firefox, LibreOffice, gestionnaire de fichiers). En haut, une barre noire avec l'heure au centre, les icônes système à droite (réseau, volume, batterie). Cliquez sur Activities en haut à gauche pour voir la vue d'ensemble avec toutes vos fenêtres ouvertes et la zone de recherche. Essayez de déplacer le pointeur dans le coin supérieur gauche : la vue Activities s'active automatiquement.

Avant de commencer vos TP, installez les Additions Invité VirtualBox pour améliorer l'intégration entre la VM et votre système hôte. Dans la fenêtre VirtualBox (menu du haut), cliquez sur Périphériques > Insérer l'image CD des Additions Invité. Une notification apparaît dans Ubuntu vous demandant d'exécuter le logiciel. Cliquez sur Exécuter. Saisissez votre mot de passe utilisateur quand demandé. Un terminal s'ouvre et compile les additions. Attendez que le message Press Return to close this window apparaisse, puis appuyez sur Entrée. Redémarrez la VM avec `sudo reboot` dans le terminal. Après le redémarrage, vous pouvez redimensionner la fenêtre VirtualBox et Ubuntu s'adapte automatiquement (résolution dynamique). Vous pouvez activer le presse-papier partagé et le glisser-déposer entre hôte et VM dans les paramètres de la VM. Félicitations, votre environnement Ubuntu est opérationnel et optimisé pour l'apprentissage !

1.3 Découverte de l'Interface GNOME et du Terminal

1.3.1 Navigation dans l'Interface Graphique GNOME

GNOME (GNU Network Object Model Environment) est l'environnement de bureau par défaut d'Ubuntu depuis la version 17.10. C'est l'interface graphique que vous voyez : fenêtres, icônes, menus, barre de titre. GNOME se veut simple, élégant, et épuré. Le dock latéral gauche, appelé dash, contient vos applications favorites et celles en cours d'exécution (un point lumineux apparaît sous l'icône des apps ouvertes). Pour lancer une application, cliquez sur son icône ou utilisez la vue Activities. La barre supérieure est fixe et toujours visible : à gauche Activities, au centre la date et l'heure (cliquez dessus pour voir le calendrier et les notifications), à droite les icônes système (volume, réseau, batterie, nom d'utilisateur). Cliquer sur votre nom ouvre un menu avec des options rapides : Paramètres, Verrouiller, Déconnexion, Éteindre.

Ouvrez l'application Fichiers (icône de dossier dans le dock). C'est le gestionnaire de fichiers, équivalent de l'Explorateur Windows. La barre latérale gauche montre les emplacements importants : Personnel (votre dossier home /home/etudiant), Bureau (/home/etudiant/Desktop où apparaissent les fichiers que vous mettez sur le bureau), Téléchargements, Documents, Images, Vidéos, Musique, et les volumes montés (disques, clés USB). Le panneau principal affiche le contenu du dossier sélectionné. Vous pouvez créer des dossiers (clic droit > Nouveau dossier), renommer, déplacer, copier des fichiers. Essayez de créer un dossier TP-Linux sur votre Bureau : ouvrez Fichiers, cliquez sur Bureau dans la barre latérale, clic droit dans l'espace vide > Nouveau dossier, nommez-le TP-Linux, appuyez sur Entrée. Ce dossier apparaît sur votre bureau graphique. Cette manipulation simple illustre comment l'interface graphique est une couche visuelle par-dessus les fichiers réels du système.

1.3.2 Lancement et Utilisation du Terminal

Le terminal est LA fenêtre sur la puissance de Linux. C'est une interface textuelle où vous tapez des commandes que le système exécute immédiatement. Pour de nombreux étudiants venant de Windows, le terminal semble archaïque et intimidant. Pourtant, c'est l'outil le plus puissant d'un administrateur système Linux : rapide, précis, automatisable, et accessible à distance via SSH même depuis un téléphone. Apprendre le terminal, c'est gagner en efficacité et en compréhension profonde du système. Pour l'ouvrir, plusieurs méthodes : (1) Cliquez sur Activities, tapez terminal, puis cliquez sur l'icône Terminal. (2) Appuyez sur

Ctrl+Alt+T (raccourci clavier universel Ubuntu). (3) Clic droit sur le bureau > Ouvrir dans un terminal. Une fenêtre noire (ou blanche selon le thème) s'ouvre avec une ligne de texte se terminant par \$: c'est le prompt, l'invite de commande qui attend vos instructions.

Décryptons ce qui est affiché. Le prompt typique ressemble à : `etudiant@ubuntu-vm:~$` où `etudiant` est votre nom d'utilisateur, `ubuntu-vm` est le nom de votre machine (hostname), le symbole `@` signifie « sur », `~` représente votre répertoire personnel (home directory, équivalent de `C:\Users\VotreNom` sous Windows), et `$` indique que vous êtes un utilisateur normal (si vous voyez `#`, c'est que vous êtes root, l'administrateur tout-puissant). Tapez votre première commande : `pwd` (Print Working Directory = affiche le répertoire courant) puis appuyez sur Entrée. Le terminal affiche `/home/etudiant`, c'est le chemin complet de votre emplacement actuel dans l'arborescence des fichiers. Tapez maintenant `ls` (list = liste) : le terminal affiche les noms des fichiers et dossiers dans votre répertoire personnel : Bureau, Documents, Téléchargements, Images, etc. Vous venez d'exécuter vos deux premières commandes Linux ! Notez que Linux est sensible à la casse : `ls` fonctionne, mais `LS` ou `Ls` produisent une erreur `command not found`. Tout est en minuscules par convention.

1.3.3 Anatomie d'une Commande Linux

Une commande Linux suit toujours la même structure :

COMMANDE [OPTIONS] [ARGUMENTS].

La **COMMANDE** est le nom du programme à exécuter (`ls`, `cd`, `mkdir`, etc.).

Les **OPTIONS** modifient le comportement de la commande et commencent généralement par un tiret - (option courte d'une lettre) ou deux tirets -- (option longue avec un mot complet).

Les **ARGUMENTS** sont les cibles sur lesquelles la commande agit, souvent des noms de fichiers ou de répertoires. Exemple : `ls -l /etc` décomposé donne : `ls` est la commande (lister), `-l` est une option (format long avec détails), et `/etc` est l'argument (le répertoire à lister). Tapez cette commande et observez : au lieu d'une simple liste de noms, vous obtenez des colonnes avec permissions, propriétaire, taille, date de modification pour chaque fichier du répertoire `/etc`.

Les options peuvent se combiner. `ls -l -a` liste les fichiers en format long ET inclut les fichiers cachés (ceux dont le nom commence par un point). On peut raccourcir en `ls -la`, les options courtes s'enchaînent sans espace. Essayez : tapez `ls -la` dans votre home. Vous voyez

maintenant des entrées comme `.bashrc`, `.profile`, `.config` qui sont des fichiers et dossiers de configuration cachés (le point au début les rend invisibles dans l'affichage normal).

Certaines commandes acceptent des options longues pour plus de clarté : `ls --all` est équivalent à `ls -a`. Quand vous apprenez une nouvelle commande, consultez son manuel intégré avec `man` suivi du nom de la commande. Exemple : `man ls` affiche le manuel complet de `ls` avec toutes ses options expliquées en détail. Utilisez les flèches haut/bas pour naviguer, `q` pour quitter. La maîtrise du terminal passe par la pratique régulière de ces commandes de base et la lecture attentive de leur documentation.

TRAVAIL PRATIQUE 1 : Installation et Prise en Main d'Ubuntu

Objectifs du TP 1

- Installer Ubuntu 22.04 LTS dans VirtualBox de façon autonome
- Naviguer dans l'interface GNOME et personnaliser le bureau
- Ouvrir un terminal et exécuter des commandes de base
- Créer des dossiers et fichiers depuis le terminal
- Comprendre le système de fichiers et les chemins absolus/relatifs

Exercice 1.1 : Installation Complète

Suivez scrupuleusement toutes les étapes vues en cours pour installer Ubuntu 22.04 dans VirtualBox. Créez une VM avec 4 Go de RAM et 30 Go de disque. Installez Ubuntu en sélectionnant English comme langue et French comme disposition clavier. Créez un utilisateur avec votre prénom en minuscules. Après l'installation, prenez un snapshot de la VM que vous nommerez Installation-Propre pour pouvoir y revenir en cas de problème.

Exercice 1.2 : Navigation Graphique

1. Créez trois dossiers sur votre Bureau : Cours, TP, Projets
2. Dans le dossier Cours, créez trois sous-dossiers : Linux, Reseaux, Programmation
3. Ouvrez l'application Gestionnaire de fichiers et naviguez dans votre dossier Personnel
4. Trouvez le dossier Téléchargements et comptez combien de fichiers il contient
5. Changez le fond d'écran en faisant clic droit sur le bureau > Changer l'arrière-plan

Exercice 1.3 : Premières Commandes Terminal

6. Ouvrez un terminal avec `Ctrl+Alt+T`

7. Tapez la commande `whoami` et notez le résultat (votre nom d'utilisateur)
8. Tapez `pwd` pour afficher votre répertoire courant (devrait être `/home/votre_nom`)
9. Tapez `ls` pour lister le contenu de votre répertoire personnel
10. Tapez `ls -l` pour voir les détails (permissions, tailles, dates)
11. Tapez `ls -la` pour voir aussi les fichiers cachés (commençant par un point)
12. Tapez `date` pour afficher la date et l'heure système
13. Tapez `cal` pour afficher le calendrier du mois en cours
14. Tapez `clear` pour nettoyer l'écran du terminal
15. Tapez `history` pour voir l'historique de vos commandes

Exercice 1.4 : Création de Fichiers et Dossiers

16. Dans le terminal, tapez `cd Bureau` pour aller dans votre dossier Bureau
17. Tapez `pwd` pour confirmer que vous êtes dans `/home/votre_nom/Bureau`
18. Créez un dossier avec `mkdir TP1-Linux`
19. Entrez dans ce dossier avec `cd TP1-Linux`
20. Créez un fichier texte vide avec `touch notes.txt`
21. Vérifiez que le fichier existe avec `ls -l`
22. Créez trois sous-dossiers d'un coup : `mkdir exercice1 exercice2 exercice3`
23. Listez le contenu pour voir vos trois dossiers
24. Revenez au dossier Bureau avec `cd ..`
25. Listez le Bureau et confirmez que TP1-Linux apparaît

CHAPITRE II : SYSTEME DE FICHIERS ET NAVIGATION

2.1 L'Arborescence du Système de Fichiers Linux (FHS)

2.1.1 Introduction au Filesystem Hierarchy Standard

Sous Linux, tous les fichiers et répertoires s'organisent selon une structure arborescente unique et normalisée appelée FHS (Filesystem Hierarchy Standard). Contrairement à Windows qui utilise des lettres de lecteur (C:\, D:\), Linux part d'un unique point d'entrée appelé la racine, représentée par le symbole /. Depuis cette racine, tout le système se ramifie comme un arbre : chaque répertoire peut contenir des fichiers et d'autres répertoires (appelés sous-répertoires), formant ainsi une hiérarchie complète. Cette organisation n'est pas aléatoire : elle suit une norme internationale (le FHS, maintenu par la Linux Foundation) qui définit précisément l'emplacement de chaque catégorie de fichiers. Comprendre cette arborescence est fondamental pour administrer un système Linux car vous devez savoir où chercher les fichiers de configuration, où sont installés les programmes, où se trouvent les journaux système, et comment s'organise l'espace utilisateur.

Une analogie puissante pour vos étudiants : imaginez une immense bibliothèque universitaire. Le bâtiment entier est la racine /. Chaque étage est un répertoire principal (/etc, /home, /usr). Chaque salle sur un étage est un sous-répertoire. Les livres sont les fichiers. La règle fondamentale est que tout livre doit être dans une salle, dans un étage, dans le bâtiment il ne peut pas exister « en dehors » de la bibliothèque. De même, sous Linux, chaque fichier a nécessairement un chemin complet qui part de /. Il n'y a aucune exception. Même les périphériques comme les clés USB, les disques durs, les imprimantes sont représentés comme des fichiers dans cette arborescence, généralement dans /dev ou montés dans /mnt. Cette philosophie « tout est fichier » est l'une des idées les plus élégantes et puissantes du design Unix/Linux.

2.1.2 Les Répertoires Principaux de l'Arborescence FHS

L'arborescence FHS définit avec précision le rôle de chaque répertoire racine. Voici les répertoires principaux que vous rencontrerez quotidiennement en tant qu'administrateur Linux :

Répertoire	Nom complet	Contenu et rôle	Exemple de fichiers
/	Racine (Root)	Point d'entrée unique de toute l'arborescence	bin, etc, home, usr, var...
/bin	Binaries	Commandes essentielles accessibles à tous les utilisateurs	ls, cp, mv, cat, bash
/boot	Boot	Fichiers de démarrage : noyau Linux et GRUB	vmlinuz, grub.cfg
/dev	Devices	Fichiers spéciaux représentant les périphériques matériels	sda, tty0, null, random
/etc	Et cetera	Fichiers de configuration système globaux	passwd, fstab, hostname
/home	Home	Répertoires personnels de chaque utilisateur ordinaire	etudiant/, KIIA/, bob/
/lib	Libraries	Bibliothèques partagées pour /bin et /sbin	libc.so, libm.so
/media	Media	Points de montage automatiques (clés USB, CD-ROM)	usb/, cdrom/
/mnt	Mount	Points de montage temporaires manuels	disque_externe/
/opt	Optional	Logiciels optionnels installés manuellement	google/, discord/
/proc	Processes	Système de fichiers virtuel : état du noyau et processus	1/, cpuinfo, meminfo
/root	Root home	Répertoire personnel de l'utilisateur root (administrateur)	.bashrc, .ssh/
/sbin	System Bin	Commandes système réservées à l'administrateur root	fdisk, iptables, mount

Répertoire	Nom complet	Contenu et rôle	Exemple de fichiers
/srv	Services	Données des services hébergés (web, FTP...)	www/, ftp/
/sys	System	Interface avec le noyau (périphériques, pilotes)	block/, bus/, class/
/tmp	Temporary	Fichiers temporaires effacés au redémarrage	session_cache, locks
/usr	Unix Sys. Resources	Programmes et données des utilisateurs (non-essentiels)	bin/, lib/, share/
/var	Variable	Fichiers variables : logs, bases de données, mails	log/, spool/, cache/

2.1.3 Focus sur les Répertoires Critiques pour l'Administration

Le répertoire /etc : le cœur de la configuration

Le répertoire /etc (prononcer « et-cé ») est l'un des plus importants pour un administrateur système. Il contient tous les fichiers de configuration globaux du système et des services installés. Chaque fois que vous modifiez le comportement du système (réseau, utilisateurs, services, sécurité), vous intervenez dans /etc.

Les fichiers ici sont principalement des fichiers texte lisibles, ce qui est une philosophie Unix fondamentale : la configuration doit être transparente, lisible et éditée avec un simple éditeur de texte.

Quelques fichiers incontournables :

/etc/passwd contient la liste de tous les comptes utilisateurs,

/etc/fstab définit les systèmes de fichiers montés automatiquement au démarrage,

/etc/hostname stocke le nom de la machine,

/etc/hosts associe des noms à des adresses IP,

et **/etc/apt/sources.list** (sous Ubuntu/Debian) définit les dépôts de paquets.

Le répertoire /var : les fichiers qui changent

Le répertoire /var (variable) contient tous les fichiers dont le contenu change fréquemment pendant le fonctionnement normal du système. C'est ici que vous trouverez les journaux système (logs) dans /var/log, les files d'attente d'impression dans /var/spool/cups, les bases de données des applications dans /var/lib, et les fichiers de cache dans /var/cache. Pour un administrateur, /var/log est particulièrement critique : c'est là que se trouvent les traces de tout ce qui se passe sur le système. Le fichier /var/log/syslog enregistre les messages système généraux, /var/log/auth.log trace toutes les tentatives de connexion (réussies ou non), /var/log/apt/history.log conserve l'historique des installations de paquets. En cas de problème, commencer par examiner ces logs est souvent la première étape du diagnostic.

Le répertoire /proc : la fenêtre sur le noyau

Le répertoire /proc est fascinant car il ne correspond à aucun fichier réel sur disque : c'est un système de fichiers virtuel généré en temps réel par le noyau Linux. Chaque processus en cours d'exécution y est représenté par un sous-répertoire numéroté (le PID du processus). Par exemple, /proc/1 contient des informations sur le processus init (le premier processus lancé au démarrage). /proc/cpuinfo révèle les caractéristiques précises de votre processeur, /proc/meminfo affiche l'état de la mémoire RAM en temps réel, /proc/mounts liste tous les systèmes de fichiers montés. Ces fichiers sont une mine d'informations pour le diagnostic système. La commande `cat /proc/cpuinfo` vous donnera plus de détails sur votre processeur que n'importe quel logiciel tiers.

⚠ Points d'attention pour les étudiants débutants

⚠ /root ≠ / : /root est le home de l'administrateur, / est la racine du système

⚠ /bin vs /usr/bin : sous Ubuntu moderne, /bin est un lien symbolique vers /usr/bin

⚠ Ne JAMAIS modifier /proc ou /sys directement : ces répertoires sont gérés par le noyau

⚠ /tmp est vidé à chaque redémarrage : ne jamais y stocker des fichiers importants

⚠ /etc est réservé à root en écriture : utilisez sudo pour modifier les configurations

Conseil : mémorisez d'abord /, /home, /etc, /var, /bin, /tmp — les plus utilisés quotidiennement

2.2 Chemins Absolus et Chemins Relatifs

2.2.1 Comprendre la Notion de Chemin

Un chemin (path en anglais) est l'adresse complète d'un fichier ou répertoire dans l'arborescence Linux. Comme une adresse postale permet de localiser une maison dans une ville, un chemin Linux permet de localiser précisément un fichier parmi les millions que peut contenir un système. Il existe deux types de chemins fondamentaux : les chemins absolus et les chemins relatifs. Comprendre leur différence est essentiel car toutes les commandes Linux acceptent ces deux formes et le choix de l'une ou l'autre influence l'efficacité et la lisibilité de vos commandes. Une confusion fréquente chez les débutants est de ne pas distinguer leur position actuelle dans l'arborescence (répertoire courant) et la destination souhaitée. Les chemins relatifs dépendent du répertoire courant, les chemins absolus sont indépendants de votre position.

2.2.2 Les Chemins Absolus

Un chemin absolu est un chemin qui part toujours de la racine / et décrit le chemin complet pour atteindre une ressource, quelle que soit votre position actuelle dans l'arborescence.

Il commence toujours par /. Par exemple, /home/etudiant/Documents/rapport.pdf est un chemin absolu : que vous soyez dans /, dans /tmp, dans /var/log, ou n'importe où, ce chemin désignera toujours le même fichier. L'avantage des chemins absolus est leur clarté et leur non-ambiguïté : vous savez toujours exactement de quoi vous parlez. Leur inconvénient est leur longueur : écrire /home/etudiant/Bureau/TP-Linux/exercice1/fichier.txt à chaque commande est laborieux. Dans les scripts shell, on préfère souvent les chemins absolus pour garantir que le script fonctionnera quel que soit le répertoire depuis lequel il est exécuté.

```
$ Terminal
```

```
# Exemples de chemins ABSOLUS (commencent toujours par /)
```

```
$ cat /etc/hostname           # Affiche le nom de la machine
```

```
$ ls /home/etudiant/Documents # Liste le dossier Documents
```

```
$ cd /var/log                 # Va dans les logs système
```

```
$ ls /usr/bin/python3         # Vérifie l'existence de Python3
```

```
# Un chemin absolu fonctionne PARTOUT, quelle que soit votre position
```

```
$ pwd                         # /tmp (vous êtes dans /tmp)
```

```
$ cat /etc/hostname           # Fonctionne quand même !
```

```
$ cd /home/etudiant          # Revient au home depuis n'importe où
```

2.2.3 Les Chemins Relatifs

Un chemin relatif décrit le chemin pour atteindre une ressource en partant du répertoire courant (là où vous êtes). Il ne commence pas par / mais par le nom d'un fichier ou répertoire, ou par des symboles spéciaux. Les deux symboles fondamentaux sont . (point) qui représente le répertoire courant, et .. (deux points) qui représente le répertoire parent (un niveau au-dessus). Par exemple, si vous êtes dans /home/etudiant, le chemin relatif Documents/rapport.pdf désigne /home/etudiant/Documents/rapport.pdf. Si vous êtes dans /home/etudiant/Documents, le chemin relatif ../Bureau désigne /home/etudiant/Bureau (on remonte d'un niveau avec .. puis on descend dans Bureau). Les chemins relatifs sont plus courts et plus pratiques pour naviguer dans des arborescences proches, mais ils ne fonctionnent que si vous êtes au bon endroit.

```
$ Terminal

# Exemples de chemins RELATIFS (dépendent du répertoire courant)

$ pwd                # /home/etudiant (position actuelle)
$ ls Documents       # Liste /home/etudiant/Documents
$ cd Bureau          # Va dans /home/etudiant/Bureau
$ cd ..              # Remonte à /home/etudiant
$ cd ../KIIAce       # Va dans /home/KIIA (sibling)
$ cd ../../etc        # Remonte 2 niveaux puis descend dans etc

# Symboles spéciaux importants

$ cd ~               # ~ = votre répertoire HOME (/home/etudiant)
$ cd ~/Documents     # Chemin depuis le HOME (absolu-relatif)
$ cd -               # Retourne au répertoire précédent
$ ls .               # . = répertoire courant (ls . = ls)
$ cp fichier.txt ./copie.txt # Copier dans le répertoire courant
```

Symbole	Signification	Exemple	Résultat si on est dans /home/etudiant
/	Racine du système	ls /	Liste tout le contenu de la racine
~	Répertoire home de	cd ~	Retourne à /home/etudiant

Symbole	Signification	Exemple	Résultat si on est dans /home/etudiant
	l'utilisateur		
.	Répertoire courant	ls .	Liste /home/etudiant (idem que ls)
..	Répertoire parent	cd ..	Va dans /home
../..	Deux niveaux au-dessus	cd ../..	Va dans / (la racine)
-	Répertoire précédent	cd -	Retourne là où on était avant

Analogie : GPS vs Itinéraire à pied

Chemin ABSOLU = Adresse GPS complète : "Avenue Kennedy, Bastos, Yaoundé, Cameroun"

→ Fonctionne depuis n'importe où dans le monde, sans ambiguïté

Chemin RELATIF = Directions depuis où vous êtes : "2ème rue à gauche, puis tout droit"

→ Ne fonctionne QUE si vous partez du bon endroit !

Symbole ~ = "Mon domicile" : quelle que soit votre position, ~ vous ramène chez vous

Symbole .. = "Sortez de la pièce actuelle, allez dans le couloir du dessus"

2.3 Commandes Essentielles de Navigation

2.3.1 La commande pwd

pwd (Print Working Directory) est la commande la plus simple mais aussi l'une des plus utilisées. Elle affiche le chemin absolu complet du répertoire dans lequel vous vous trouvez actuellement. Quand vous êtes perdu dans l'arborescence, pwd est votre boussole : il vous dit exactement où vous êtes. Cette commande ne prend généralement aucun argument et

n'a que peu d'options. Son usage est systématique lors des premiers pas en terminal : tapez `pwd` après chaque `cd` pour vérifier que vous êtes bien où vous pensez être.

```
$ Terminal

$ pwd
/home/etudiant

$ cd /var/log
$ pwd
/var/log

$ cd ../../tmp
$ pwd
/tmp

# Option utile : -P (physical) pour résoudre les liens symboliques
$ pwd -P      # Affiche le chemin réel, pas le lien symbolique
```

2.3.2 La commande `cd`

`cd` (Change Directory) est la commande de navigation principale. Elle change votre répertoire courant. Maîtriser `cd` avec ses différentes formes est essentiel pour travailler efficacement dans un terminal Linux. La commande `cd` accepte aussi bien des chemins absolus que relatifs, et comprend les symboles spéciaux `~`, `..`, et `-`. Une subtilité importante : `cd` sans argument vous ramène toujours dans votre répertoire home, exactement comme `cd ~`. C'est un raccourci précieux quand vous êtes perdus en profondeur dans l'arborescence et voulez revenir rapidement à votre espace de travail habituel.

```
$ Terminal

# Formes de base

$ cd /etc           # Chemin absolu : va dans /etc
$ cd Documents      # Chemin relatif : Documents dans le dossier courant
$ cd ..             # Remonte d'un niveau (répertoire parent)
$ cd ../../..       # Remonte deux niveaux
$ cd ~              # Retour au répertoire home
$ cd                # Sans argument : retour au home (identique à cd ~)
$ cd -              # Revient au répertoire précédent (très pratique !)
```



```
# Exemples concrets
$ cd /home/etudiant/Bureau/TP-Linux      # Chemin absolu complet
$ cd ~/Bureau/TP-Linux                   # Même chose avec ~
$ cd TP-Linux                            # Relatif si on est déjà dans ~/Bureau

# Astuces avancées
$ cd /var && ls                          # && : enchaîne deux commandes
$ pushd /tmp                             # Mémoire position actuelle et va dans /tmp
$ popd                                    # Retourne à la position mémorisée
```

2.3.3 La commande ls

ls (List) affiche le contenu d'un répertoire. C'est probablement la commande la plus utilisée en Linux. Sans argument, elle liste le contenu du répertoire courant. Avec un chemin en argument, elle liste le contenu du répertoire spécifié. ls possède de nombreuses options qui transforment l'affichage et révèlent des informations supplémentaires. La maîtrise des options principales de ls vous permettra d'obtenir rapidement des informations précises sur les fichiers : permissions, taille, propriétaire, date de modification.

Option	Signification	Description détaillée
-l	Long format	Affiche les détails : permissions, propriétaire, taille, date, nom
-a	All	Inclut les fichiers cachés (noms commençant par un point)
-h	Human-readable	Affiche les tailles en Ko, Mo, Go (lisibles) avec -l
-R	Recursive	Liste récursivement tous les sous-répertoires
-t	Time	Trie par date de modification (plus récent en premier)
-S	Size	Trie par taille (plus grand en premier)
-r	Reverse	Inverse l'ordre de tri
--color	Color	Colore la sortie selon le type (répertoires en bleu, exe en vert)

Option	Signification	Description détaillée
-l	One per line	Un fichier par ligne (utile pour les scripts)
-d	Directory	Affiche les infos du répertoire lui-même, pas son contenu

```

$ Terminal

$ ls                                # Liste le répertoire courant
$ ls /etc                           # Liste le contenu de /etc
$ ls -l                             # Format long (détails)
$ ls -la                            # Long + fichiers cachés
$ ls -lh                            # Long + tailles lisibles (Ko, Mo)
$ ls -lha                           # Combine : long + cachés + lisible
$ ls -lt                            # Trié par date (plus récent d'abord)
$ ls -ls                             # Trié par taille (plus grand d'abord)
$ ls -R ~/Documents                 # Récursif : tout le contenu et sous-dossiers

# Lecture du format long : ls -l
# -rw-r--r-- 1 etudiant etudiant 4096 mars  4 09:30 rapport.txt
# ^^^^^^^^^^ ^^^^^^^^^ ^^^^^^^^^ ^^^^^ ^^^^^^^^^^^^^^^^^ ^^^^^^^^^^^^^
# permissions user      group   size   date/heure   nom fichier

# Filtrer avec des patterns (wildcards)
$ ls *.txt                          # Tous les fichiers .txt
$ ls rapport*                       # Tous commençant par "rapport"
$ ls /etc/*.conf                    # Tous les .conf dans /etc

```

2.3.4 La commande find

`find` est une commande extrêmement puissante qui permet de rechercher des fichiers et répertoires selon de nombreux critères : nom, type, taille, date de modification, permissions, propriétaire. Contrairement à une simple liste, `find` explore récursivement toute une arborescence et peut même exécuter des actions sur les fichiers trouvés. Sa syntaxe est : `find [répertoire de départ] [critères] [actions]`. Si vous ne spécifiez pas de répertoire, `find` commence dans le répertoire courant.

```
$ Terminal
```

```
# Recherche par NOM
```

```
$ find / -name "passwd"           # Cherche "passwd" depuis la racine
$ find /home -name "*.txt"        # Tous les .txt dans /home
$ find . -name "rapport*"         # Commence par "rapport" dans . (courant)
$ find /etc -iname "*.conf"       # -iname = insensible à la casse
```

```
# Recherche par TYPE
```

```
$ find /home -type f              # -type f = fichiers seulement
$ find /home -type d              # -type d = répertoires seulement
$ find /dev -type l               # -type l = liens symboliques
```

```
# Recherche par TAILLE
```

```
$ find / -size +100M              # Fichiers de plus de 100 Mo
$ find /tmp -size -1k             # Fichiers de moins de 1 Ko
```

```
# Recherche par DATE
```

```
$ find /var/log -mtime -1         # Modifiés dans les 24 dernières heures
$ find /home -newer /tmp/ref.txt   # Plus récents que ref.txt
```

```
# Combiner des critères avec -and, -or, -not
```

```
$ find / -name "*.log" -size +10M -type f # .log ET > 10Mo ET fichier
```

```
# Exécuter une action sur les résultats avec -exec
```

```
$ find /tmp -name "*.tmp" -exec rm {} ;   # Supprimer les .tmp dans /tmp
$ find /home -name "*.txt" -exec ls -lh {} ; # Détails de tous les .txt
```

2.3.5 La commande locate

locate est une alternative à find pour la recherche de fichiers, mais elle fonctionne différemment : au lieu d'explorer le système de fichiers en temps réel, elle interroge une base de données pré-indexée. Cela la rend extrêmement rapide (des milliers de fois plus rapide que find sur un grand système), mais avec une limitation importante : la base de données n'est pas mise à jour en temps réel. Elle est généralement actualisée une fois par jour via une tâche cron. Pour forcer la mise à jour, utilisez la commande `sudo updatedb`. locate est idéale pour retrouver rapidement des fichiers système existants depuis longtemps, mais find reste

nécessaire pour trouver des fichiers récemment créés ou pour des recherches avec des critères complexes.

```
$ Terminal

# Installation si nécessaire (Ubuntu)
$ sudo apt install mlocate

# Mettre à jour la base de données
$ sudo updatedb

# Utilisation de base
$ locate passwd          # Tous les fichiers contenant "passwd" dans le
nom
$ locate /etc/*.conf      # Les .conf dans /etc (depuis la base de données)
$ locate -i README        # -i : insensible à la casse
$ locate -n 10 python     # -n : limiter à 10 résultats
$ locate -c passwd        # -c : compte le nombre de résultats

# Comparaison find vs locate
# find / -name "*.log"    → Explore tout le disque en temps réel (lent mais à
jour)
# locate "*.log"          → Interroge la base de données (ultra-rapide mais peut
être obsolète)
```

2.3.6 La commande tree

tree est une commande optionnelle (non installée par défaut) qui affiche le contenu d'un répertoire sous forme d'arborescence graphique, ce qui est très pratique pour comprendre la structure d'un projet ou d'un répertoire système. Elle représente visuellement la hiérarchie avec des lignes et des branches, rendant immédiatement lisible la profondeur et l'organisation des fichiers. C'est un outil pédagogique particulièrement utile pour les étudiants qui apprennent la structure du système de fichiers Linux.

```
$ Terminal

# Installation
$ sudo apt install tree

# Utilisation
```

```
$ tree # Arborescence depuis le répertoire courant
$ tree /etc # Arborescence de /etc
$ tree -L 2 # Limite à 2 niveaux de profondeur
$ tree -d # Affiche uniquement les répertoires
$ tree -h # Affiche les tailles des fichiers
$ tree -a # Inclut les fichiers cachés

# Exemple de sortie de tree -L 2 /home/etudiant
# /home/etudiant
# └─ Bureau
#   └─ TP-Linux
# └─ Documents
#     └─ Cours
#     └─ Projets
# └─ Téléchargements
# └─ .bashrc
# 4 directories, 1 file
```

TRAVAIL PRATIQUE 2 : Exploration du Système de Fichiers et Navigation

Objectifs du TP 2

- Explorer l'arborescence FHS et identifier le rôle de chaque répertoire
- Maîtriser la navigation avec `cd`, `ls`, `pwd` en utilisant chemins absolus et relatifs
- Utiliser `find` et `locate` pour rechercher des fichiers dans le système
- Comprendre et utiliser les symboles spéciaux `~`, `.`, `..` et `-`
- Créer et organiser une arborescence de projet personnelle

Exercice 2.1 : Exploration de l'Arborescence FHS

Répondez aux questions suivantes en exécutant les commandes appropriées dans le terminal :

N°	Question	Commande attendue
1	Listez tous les répertoires présents directement dans la racine /	<code>ls /</code>
2	Combien de fichiers se trouvent dans /etc ? (hint	<code>ls /etc wc -l</code>

N°	Question	Commande attendue
	: ls wc -l)	
3	Quel est le contenu de /etc/hostname ?	cat /etc/hostname
4	Listez les répertoires dans /home	ls /home
5	Affichez les informations du processeur avec cat /proc/cpuinfo	cat /proc/cpuinfo head -20
6	Quel espace disque total utilise /var/log ? (hint : du -sh)	sudo du -sh /var/log
7	Installez et utilisez tree -L 1 / pour visualiser la racine	sudo apt install tree && tree -L 1 /
8	Affichez la mémoire RAM disponible depuis /proc/meminfo	cat /proc/meminfo grep MemTotal

Exercice 2.2 : Navigation avec Chemins Absolus et Relatifs

Exécutez les commandes suivantes dans l'ordre. Avant chaque commande, prédisez ce qui va se passer, puis vérifiez avec pwd.

```
$ Terminal

# Partie A : Chemins absolus

$ cd /           # Question : où êtes-vous ? Vérifiez avec pwd
$ cd /home       # Allez dans /home
$ cd /etc        # Allez dans /etc
$ ls -l          # Listez /etc en format long
$ cd /var/log    # Allez dans /var/log
$ ls -lh         # Listez avec tailles lisibles

# Partie B : Chemins relatifs

$ cd /home       # Repartez de /home
$ cd etudiant    # Relatif : va dans /home/etudiant
$ cd Bureau      # Relatif : va dans ~/Bureau
```

```
$ cd ..          # Remonte à /home/etudiant
$ cd ../../      # Remonte à /home puis à / - vérifiez !

# Partie C : Symboles spéciaux
$ cd ~           # Retour au home
$ cd /tmp        # Allez dans /tmp
$ cd -           # Revient au home - vérifiez avec pwd !
$ cd ~/Bureau    # Combine ~ et chemin relatif
$ pwd            # Doit afficher /home/etudiant/Bureau
```

Exercice 2.3 : Recherche avec find et locate

Effectuez les recherches suivantes et notez le nombre de résultats obtenus :

6. Trouvez tous les fichiers .conf dans /etc

```
$ Terminal
```

```
$ find /etc -name « *.conf » -type f
```

2. Cherchez le fichier « passwd » dans tout le système

```
$ Terminal
```

```
$ find / -name « passwd » 2>/dev/null
```

3. Listez tous les répertoires dans /home/etudiant

```
$ Terminal
```

```
$ find /home/etudiant -type d
```

4. Trouvez les fichiers modifiés dans les dernières 24h dans /var/log

```
$ Terminal
```

```
$ find /var/log -mtime -1 -type f
```

5. Cherchez les fichiers de plus de 1 Mo dans /var

```
$ Terminal
```

```
$ find /var -size +1M -type f 2>/dev/null
```

6. Utilisez locate pour trouver tous les fichiers python3

```
$ Terminal
```

```
$ locate python3
```

Exercice 2.4 :Création d'une Arborescence de Projet (25 min)

Créez depuis le terminal la structure suivante dans votre répertoire home :

```
$ Terminal

# Structure à créer :
~/KIIA-Linux/
├─ Module1/
│   └─ cours/
│       :,,,,,,,,,,,,,,,,,,,,,|   └─ tp/
│       └─ corrections/
├─ Module2/
│   └─ cours/
│   └─ tp/
│   └─ corrections/
└─ Ressources/
    └─ commandes.txt
    └─ glossaire.txt

# Commandes pour créer cette structure
$ cd ~
$ mkdir -p KIIA-Linux/Module1/{cours,tp,corrections}
$ mkdir -p KIIA-Linux/Module2/{cours,tp,corrections}
$ mkdir -p KIIA-Linux/Ressources
$ touch KIIA-Linux/Ressources/commandes.txt
$ touch KIIA-Linux/Ressources/glossaire.txt

# Vérification avec tree
$ tree ~/KIIA-Linux

# Naviguer dans la structure avec des chemins relatifs et absolus
$ cd ~/KIIA-Linux/Module2/tp
$ pwd
$ cd ../../Module1/cours
$ pwd
$ cd ~/KIIA-Linux/Ressources
$ echo "pwd, ls, cd, find, locate, tree" > commandes.txt
$ cat commandes.txt
```


CHAPITRE III : GESTION DES FICHIERS ET PERMISSIONS

3.1 Manipulation des Fichiers et Répertoires

3.1.1 Créer, Copier et Déplacer des Fichiers

La gestion quotidienne des fichiers sous Linux repose sur un ensemble restreint de commandes très puissantes. Contrairement à l'interface graphique où l'on glisse-dépose les fichiers, le terminal offre une précision et une rapidité incomparables, surtout pour traiter des centaines ou des milliers de fichiers en une seule commande. Les quatre opérations fondamentales sont : créer (touch, mkdir), copier (cp), déplacer/renommer (mv) et supprimer (rm, rmdir). Maîtriser ces commandes est un prérequis absolu pour tout administrateur système : la plupart du travail quotidien (sauvegardes, déploiements, maintenance) consiste en combinaisons de ces opérations élémentaires.

- **La commande cp**

cp (copy) copie un fichier ou un répertoire vers une destination. Sa syntaxe de base est cp source destination. Si la destination est un répertoire existant, cp y place la copie en conservant le nom d'origine. Si la destination est un chemin vers un fichier inexistant, cp crée la copie avec ce nouveau nom. Pour copier un répertoire entier avec tout son contenu, l'option -r (recursive) est obligatoire. L'option -p (preserve) conserve les métadonnées du fichier original : permissions, propriétaire et date de modification. Ces options sont cruciales lors des sauvegardes pour ne pas altérer les droits d'accès.

\$ **Terminal**

Copie de fichiers simples

\$ cp fichier.txt copie.txt

Copie dans le répertoire courant

\$ cp /etc/passwd /tmp/passwd.bak

Copie avec chemin absolu

\$ cp rapport.txt ~/Documents/
conservé)

Copie vers un répertoire (nom

Options importantes

\$ cp -r /home/etudiant/projet /tmp/
entier)

-r : copie RÉCURSIVE (répertoire

\$ cp -p fichier.txt sauvegarde.txt

-p : préserve permissions + dates

\$ cp -rp /var/www /backup/www-backup
complète)

Combine -r et -p (sauvegarde

\$ cp -i fichier.txt existant.txt

-i : demande confirmation si
écrasement

```
$ cp -v fichier.txt /tmp/           # -v : mode verbeux (affiche ce qui se passe)

# Copier plusieurs fichiers vers un répertoire
$ cp fichier1.txt fichier2.txt fichier3.txt /tmp/
$ cp *.txt /home/etudiant/Documents/ # Copie tous les .txt
```

■ La commande mv

mv (move) sert à la fois à déplacer et à renommer des fichiers et répertoires. Contrairement à cp, mv ne crée pas de copie : le fichier est physiquement déplacé (ou son nom est modifié dans le système de fichiers). Pour renommer un fichier, on utilise mv ancien_nom nouveau_nom dans le même répertoire. Pour déplacer, mv source destination_répertoire. Une subtilité importante : mv est instantané quand source et destination sont sur le même système de fichiers (il change simplement le nom dans le répertoire), mais il copie puis supprime si source et destination sont sur des systèmes de fichiers différents (par exemple de /home vers /tmp sur une partition séparée). Contrairement à cp, mv fonctionne directement sur les répertoires sans option -r.

```
$ Terminal

# Renommer un fichier (source et destination dans le même répertoire)
$ mv rapport.txt rapport_final.txt      # Renomme le fichier
$ mv ancien_dossier/ nouveau_dossier/    # Renomme un répertoire

# Déplacer des fichiers
$ mv rapport.txt ~/Documents/            # Déplace vers Documents
$ mv /tmp/fichier.log /var/log/          # Déplace avec chemin absolu
$ mv *.jpg ~/Images/                    # Déplace tous les .jpg

# Options utiles
$ mv -i fichier.txt /tmp/                # -i : confirme si écrasement
$ mv -v dossier/ /backup/                # -v : mode verbeux
$ mv -n source.txt dest.txt              # -n : ne jamais écraser (no-clobber)

# Déplacer + renommer en une seule commande
$ mv ~/Bureau/brouillon.txt ~/Documents/rapport_final.txt
```

■ La commande rm

rm (remove) supprime des fichiers et répertoires. C'est une commande à utiliser avec une grande prudence : sous Linux, il n'y a pas de corbeille depuis le terminal. Un fichier

supprimé avec `rm` est définitivement perdu (sauf si vous avez une sauvegarde). Cette caractéristique est à la fois une force (efficacité) et un danger (irréversibilité). La règle d'or est de toujours vérifier votre commande avant d'appuyer sur Entrée, surtout avec des wildcards. L'option `-i` (interactive) oblige `rm` à demander confirmation pour chaque fichier : recommandée pour les débutants. L'option `-r` (recursive) supprime récursivement un répertoire et tout son contenu.

```
$ Terminal

# Suppression de fichiers
$ rm fichier.txt                # Supprime un fichier (IRRÉVERSIBLE)
$ rm fichier1.txt fichier2.txt  # Supprime plusieurs fichiers
$ rm *.tmp                      # Supprime tous les .tmp du répertoire
                                courant

# Suppression de répertoires
$ rmdir dossier_vide/           # Supprime un répertoire VIDE seulement
$ rm -r dossier/                # Supprime un répertoire et tout son
                                contenu
$ rm -rf dossier/               # -f : force (pas de confirmation, pas
                                d'erreur)

# Options de sécurité
$ rm -i fichier.txt             # -i : demande confirmation
$ rm -iv *.txt                  # Confirmation + verbeux

# !! DANGER !! Ces commandes sont CATASTROPHIQUES
# rm -rf /                      → Supprime TOUT le système (désactivé sur Ubuntu moderne)
# rm -rf ~/*                    → Supprime TOUT votre home (très dangereux)
# rm -rf .*                     → Supprime les fichiers cachés incluant .bashrc !
```

⚠ Règles de sécurité absolues avec rm

⚠ Relisez TOUJOURS la commande `rm` avant d'appuyer sur Entrée

⚠ Utilisez `rm -i` en mode apprentissage pour confirmer chaque suppression

⚠ Testez d'abord avec `ls` ce que vos wildcards sélectionnent : `ls *.txt` avant `rm *.txt`

⚠ Ne jamais utiliser `rm -rf /` ou `rm -rf /*` (destruction totale du système)

⚠ Avec des wildcards (`*`, `?`), vérifiez toujours le répertoire courant avec `pwd`

Astuce : créer un alias "rm=rm -i" dans ~/.bashrc pour toujours avoir la confirmation

3.1.2 Créer et Gérer les Répertoires

La gestion des répertoires est une extension naturelle de la manipulation des fichiers. `mkdir` (make directory) crée des répertoires, et son option `-p` est particulièrement puissante car elle crée toute une arborescence en une seule commande, sans erreur si les répertoires parents existent déjà. La syntaxe d'accolades `{}` permet de créer plusieurs répertoires frères simultanément, ce qui accélère considérablement la mise en place de structures de projets. Un administrateur doit maîtriser ces raccourcis pour configurer rapidement des environnements de déploiement.

```
$ Terminal

# mkdir : créer des répertoires

$ mkdir mon_dossier                # Crée un répertoire simple
$ mkdir dossier1 dossier2 dossier3  # Crée plusieurs répertoires
$ mkdir -p projets/web/css          # -p : crée toute l'arborescence
$ mkdir -p /opt/app/{logs,config,data} # Accolades : 3 dossiers d'un coup
$ mkdir -p ~/ALI/{M1,M2,M3}/{cours,tp} # Arborescence complexe en une ligne

# Vérifier la création

$ ls -la                          # Voir les répertoires créés
$ tree ~/ALI                      # Visualiser l'arborescence (si tree
installé)

# Supprimer des répertoires

$ rmdir dossier_vide              # Seulement si le dossier est vide
$ rm -r dossier_avec_contenu      # Supprime tout le contenu

# Informations sur un répertoire

$ ls -ld /etc                    # -d : info sur le répertoire lui-même
$ du -sh ~/Documents             # Taille totale du répertoire
$ du -sh ~/Documents/*           # Taille de chaque élément
```

3.1.3 Afficher et Manipuler le Contenu des Fichiers

Lire le contenu d'un fichier texte est une opération fondamentale. Linux offre plusieurs commandes selon le besoin : `cat` pour afficher tout le contenu, `less` et `more` pour naviguer page par page dans les fichiers longs, `head` et `tail` pour voir le début ou la fin. La commande `tail -f` est particulièrement utile pour les administrateurs car elle suit un fichier en temps réel

(comme un log qui se remplit). Ces commandes sont souvent enchaînées via des pipes (|) pour filtrer, trier ou transformer les données.

\$ Terminal	
# Afficher le contenu	
\$ cat fichier.txt	# Affiche tout le fichier
\$ cat -n fichier.txt	# Avec numéros de lignes
\$ cat fichier1.txt fichier2.txt	# Affiche les deux fichiers à la suite
# Navigation dans les fichiers longs	
\$ less /var/log/syslog	# Navigue : flèches, PgUp/PgDn, q pour quitter
\$ more /etc/passwd	# Similaire à less (moins de fonctionnalités)
# Début et fin de fichier	
\$ head /etc/passwd	# 10 premières lignes (défaut)
\$ head -20 /etc/passwd	# Les 20 premières lignes
\$ tail /var/log/syslog	# 10 dernières lignes
\$ tail -50 /var/log/auth.log	# 50 dernières lignes
\$ tail -f /var/log/syslog	# -f : SUIV le fichier en temps réel (Ctrl+C pour stop)
# Créer et écrire dans des fichiers	
\$ echo "Bonjour Linux" > fichier.txt	# Écrase le fichier (crée si inexistant)
\$ echo "Nouvelle ligne" >> fichier.txt	# Ajoute à la fin (append)
\$ cat > notes.txt	# Saisie multi-ligne (Ctrl+D pour terminer)
# Compter les lignes, mots, caractères	
\$ wc fichier.txt	# Lignes, mots, octets
\$ wc -l /etc/passwd	# Compte le nombre de lignes
\$ wc -w rapport.txt	# Compte le nombre de mots

3.2 Comprendre les Permissions Linux

3.2.1 Le Modèle de Permissions Unix/Linux

Le système de permissions de Linux est l'un des fondements de sa sécurité. Chaque fichier et répertoire possède un ensemble de permissions qui contrôlent qui peut faire quoi avec ce fichier. Ce modèle repose sur trois concepts : les types d'accès (lecture, écriture,

exécution), les classes d'utilisateurs (propriétaire, groupe, autres), et la combinaison de ces deux dimensions. Ce modèle simple mais puissant, hérité d'Unix dans les années 1970, est toujours au cœur des systèmes Linux modernes. Comprendre les permissions est indispensable pour sécuriser un serveur : une permission mal configurée peut exposer des données confidentielles ou permettre des escalades de privilèges.

Analogie : la résidence universitaire KIJA

Imaginez une résidence universitaire avec des chambres, des salles communes et des bureaux :

Propriétaire (user) = Le résident de la chambre : a toutes les clés de sa chambre

Groupe (group) = Les résidents du même couloir : peuvent utiliser la salle commune partagée

Autres (others) = Les visiteurs extérieurs : accès limité aux espaces publics uniquement

Permission r (read) = Pouvoir LIRE ce qui est affiché (voir le contenu)

Permission w (write) = Pouvoir MODIFIER ou DÉPLACER (changer le contenu)

Permission x (execute) = Pouvoir ENTRER dans la pièce ou EXÉCUTER le programme

Un fichier de notes personnelles = chmod 600 : seul le propriétaire peut lire et écrire

Un script commun à l'équipe = chmod 750 : propriétaire tout, groupe lecture+exécution

3.2.2 Lire les Permissions avec ls -l

Quand vous exécutez `ls -l`, chaque ligne représente un fichier avec ses métadonnées. La première colonne, composée de 10 caractères, encode les permissions du fichier. Savoir déchiffrer cette chaîne de caractères est une compétence fondamentale que tout administrateur Linux doit posséder. Ces 10 caractères se décomposent en : 1 caractère pour le type de fichier, puis 3 groupes de 3 caractères pour les permissions du propriétaire, du groupe et des autres utilisateurs respectivement.

```
$ Terminal
```

```
# Exemple de sortie de ls -l
```

```
$ ls -l /home/etudiant/
```

```
# -rw-r--r-- 1 etudiant etudiant 4096 mars  3 10:15 rapport.txt
```

```
# drwxr-xr-x 2 etudiant etudiant 4096 mars 3 09:00 Documents/
# -rwxr-x--- 1 etudiant dev 1024 mars 3 11:30 script.sh
# lrwxrwxrwx 1 root root 10 mars 1 08:00 lien -> /etc/passwd

# Décomposition des 10 caractères :
# Position 1 : type (- = fichier, d = répertoire, l = lien symbolique)
# Positions 2-4: USER (propriétaire) : r=lecture w=écriture x=exécution
# Positions 5-7: GROUP (groupe) : r=lecture w=écriture x=exécution
# Positions 8-10: OTHERS (autres) : r=lecture w=écriture x=exécution

# Exemple : -rwxr-x---
# - rwx r-x ---
# | | | |
# | | | | Autres : aucune permission
# | | | | Groupe : lecture + exécution
# | | | | Propriét.: lecture + écriture + exécution
# | | | | Type : fichier ordinaire
```

Caractère	Position	Signification
-	1	Fichier ordinaire
d	1	Répertoire (directory)
l	1	Lien symbolique (symbolic link)
c	1	Fichier de périphérique caractère (ex : /dev/tty)
b	1	Fichier de périphérique bloc (ex : /dev/sda)
r	2,5,8	Permission de lecture (read) : voir le contenu
w	3,6,9	Permission d'écriture (write) : modifier le contenu
x	4,7,10	Permission d'exécution (execute) : lancer ou entrer dans un dossier
-	Toute	Permission absente (tiret = pas cette permission)

Caractère	Position	Signification
s	4 ou 7	SUID ou SGID (permissions spéciales, voir section 3.3)
t	10	Sticky Bit (permission spéciale, voir section 3.3)

3.2.3 Les Permissions sur les Répertoires

Les permissions ont une signification légèrement différente pour les répertoires que pour les fichiers ordinaires. Cette nuance est souvent source de confusion chez les débutants. Pour un répertoire, **r** (lecture) permet de lister son contenu avec `ls`. **w** (écriture) permet de créer, supprimer ou renommer des fichiers à l'intérieur du répertoire. **x** (exécution) permet d'entrer dans le répertoire avec `cd` et d'accéder aux fichiers qu'il contient, même si on connaît leur nom. Cette dernière permission est particulièrement subtile : un répertoire avec les permissions **r--** (lecture seule) permettrait de voir les noms des fichiers mais pas d'y entrer ni d'y accéder. Un répertoire **--x** (exécution seule) permettrait d'entrer si on connaît le nom du fichier mais pas de lister le contenu.

Permission	Sur un fichier	Sur un répertoire
r (read)	Lire le contenu du fichier (<code>cat</code> , <code>less</code> , <code>head...</code>)	Lister les fichiers avec <code>ls</code>
w (write)	Modifier, écrire dans le fichier	Créer, supprimer, renommer des fichiers dans le dossier
x (execute)	Exécuter le fichier comme programme	Entrer dans le répertoire (<code>cd</code>) et accéder aux fichiers
rw-	Lire et écrire, mais pas exécuter	Lister et modifier, mais pas entrer
r-x	Lire et exécuter, mais pas modifier	Lister et entrer, mais pas créer/supprimer
rwX	Contrôle total : lecture, écriture, exécution	Contrôle total : lister, entrer, créer, supprimer
---	Aucun accès	Aucun accès (ni lister, ni entrer)

3.3 Modifier les Permissions avec chmod

3.3.1 La Notation Symbolique de chmod

chmod (change mode) est la commande qui modifie les permissions d'un fichier ou répertoire. Il existe deux syntaxes pour spécifier les permissions : la notation symbolique (avec des lettres) et la notation octale (avec des chiffres). La notation symbolique est plus intuitive pour des modifications ciblées : on spécifie qui (u=user, g=group, o=others, a=all), l'opération (+, -, =) et les permissions concernées (r, w, x). Cette notation permet d'ajouter ou retirer une permission spécifique sans affecter les autres, ce qui est idéal pour des ajustements ponctuels.

Symbole	Signification	Exemple d'usage
u	User = propriétaire du fichier	chmod u+x script.sh
g	Group = membres du groupe propriétaire	chmod g-w fichier.txt
o	Others = tous les autres utilisateurs	chmod o-rx données.db
a	All = u + g + o (tous)	chmod a+r public.html
+	Ajouter la permission	chmod u+w notes.txt
-	Retirer la permission	chmod g-x script.sh
=	Définir exactement ces permissions	chmod u=rw fichier.txt

\$ Terminal	
# Notation SYMBOLIQUE – exemples concrets	
\$ chmod u+x script.sh	# Ajoute exécution pour le propriétaire
\$ chmod g-w fichier.txt	# Retire écriture pour le groupe
\$ chmod o-rwx confidentiel.txt	# Retire tout pour les autres
\$ chmod a+r public.html	# Ajoute lecture pour tout le monde
\$ chmod u+x,g+r script.py	# Plusieurs changements en une commande
\$ chmod u=rwx,g=rx,o= prog	# Définit exactement chaque classe

```
# Option -R : récursif (applique à tout un répertoire)
$ chmod -R 755 /var/www/html      # Applique à tout le site web
$ chmod -R u+w ~/projets          # Ajoute écriture récursivement
```

3.3.2 La Notation Octale de chmod

La notation octale représente les permissions par un nombre de 3 ou 4 chiffres. Chaque chiffre correspond à une classe d'utilisateur (propriétaire, groupe, autres) et représente la combinaison des permissions r, w, x en base 8. La logique est simple : r vaut 4, w vaut 2, x vaut 1, et on additionne les valeurs des permissions souhaitées. Ainsi, rwx = 4+2+1 = 7, rw- = 4+2 = 6, r-x = 4+1 = 5, r-- = 4, etc. Un administrateur expérimenté utilise surtout la notation octale car elle permet de définir toutes les permissions en un seul nombre compact. Connaître par cœur les combinaisons les plus courantes (755, 644, 700, 777...) est un signe de maîtrise de Linux.

Valeur	Permissions	Binaire	Signification
7	Rwx	111	Lecture + Écriture + Exécution (contrôle total)
6	rw-	110	Lecture + Écriture (pas d'exécution)
5	r-x	101	Lecture + Exécution (pas d'écriture)
4	r--	100	Lecture seule
3	-wx	011	Écriture + Exécution (rare)
2	-w-	010	Écriture seule (très rare)
1	--x	001	Exécution seule (très rare)
0	---	000	Aucune permission

```
$ Terminal
```

```
# Notation OCTALE – les combinaisons les plus courantes
```

```
$ chmod 755 script.sh      # rwxr-xr-x : standard pour exécutables publics
$ chmod 644 rapport.txt    # rw-r--r-- : standard pour fichiers texte
```

```

$ chmod 600 cle_privee      # rw----- : fichier privé (clé SSH, mot de passe)
$ chmod 700 mon_script.sh   # rwx----- : exécutable strictement privé
$ chmod 777 partage/       # rwxrwxrwx : DANGEREUX, accès total pour tous
$ chmod 750 app/            # rwxr-x--- : répertoire d'application (groupe
limité)
$ chmod 664 document.txt    # rw-rw-r-- : éditables par propriétaire et groupe

# Déchiffrer un chmod octal
# chmod 754 = 7  5  4
#           |  |  |
#           |  |  └─ Others : 4 = r-- (lecture seule)
#           |  └─── Group  : 5 = r-x  (lecture + exécution)
#           └───── User   : 7 = rwx  (contrôle total)

```

chmod octal	Permissions	Cas d'usage typique
777	Rwxrwxrwx	Répertoire temporaire partagé (à éviter en production)
755	rwxr-xr-x	Répertoires web, scripts publics, binaires système
750	rwxr-x---	Scripts d'application réservés à un groupe
700	rwx-----	Scripts privés, répertoires home
664	rw-rw-r--	Fichiers éditables par un groupe de travail
644	rw-r--r--	Fichiers texte, configurations publiques
640	rw-r-----	Fichiers de config avec données semi-sensibles
600	rw-----	Clés SSH, fichiers de mots de passe, données privées
400	r-----	Fichiers en lecture seule strictement privés

3.4 Changer la Propriété : chown et chgrp

3.4.1 La commande chown

chown (change owner) modifie le propriétaire d'un fichier ou répertoire. Cette commande est réservée à l'administrateur root (ou sudo) : un utilisateur normal ne peut pas changer le propriétaire d'un fichier, même s'il en est l'auteur actuel. Seul root peut attribuer des fichiers à d'autres utilisateurs. chown peut modifier à la fois le propriétaire et le groupe en une seule commande avec la syntaxe chown propriétaire:groupe fichier. Cette opération est fréquente lors du déploiement d'applications web : les fichiers doivent appartenir à l'utilisateur du serveur web (www-data sous Ubuntu/Apache) pour que le serveur puisse les lire.

```
$ Terminal

# Changer le propriétaire
$ sudo chown alice fichier.txt           # Donne le fichier à alice
$ sudo chown root /etc/mon_config        # Donne à root
$ sudo chown www-data /var/www/html/     # Donne au serveur web

# Changer propriétaire ET groupe simultanément
$ sudo chown alice:devs fichier.txt      # Propriétaire=alice, Groupe=devs
$ sudo chown www-data:www-data /var/www/ # Propriétaire et groupe = www-data

# Option -R : récursif (applique à tout un répertoire)
$ sudo chown -R alice:devs /home/projets/
$ sudo chown -R www-data:www-data /var/www/html/

# Vérifier le résultat
$ ls -l fichier.txt
# -rw-r--r-- 1 alice devs 1234 mars 4 10:00 fichier.txt
#
#      ^      ^
#      |      L Groupe : devs
#      L_____ Propriétaire : alice
```

3.4.2 La commande chgrp

chgrp (change group) modifie uniquement le groupe propriétaire d'un fichier, sans toucher au propriétaire. Un utilisateur peut utiliser chgrp pour changer le groupe d'un de ses fichiers, à condition d'appartenir lui-même au groupe cible. Cette commande est utile dans les environnements collaboratifs : si une équipe de développeurs partage un groupe commun (ex:

groupe "devs"), les membres peuvent attribuer leurs fichiers à ce groupe pour les rendre accessibles à leurs collègues, sans passer par root.

```
$ Terminal

# Changer uniquement le groupe
$ chgrp devs mon_script.sh           # Change le groupe en "devs"
$ sudo chgrp www-data /var/www/upload/ # Donne le groupe www-data
$ chgrp -R equipe /home/alice/projet/  # Récursif

# Équivalence : chgrp = chown :groupe
$ chgrp devs fichier.txt
$ sudo chown :devs fichier.txt        # Identique avec chown

# Voir les groupes disponibles
$ cat /etc/group                      # Liste tous les groupes du système
$ groups                             # Groupes de l'utilisateur courant
$ groups alice                        # Groupes d'un utilisateur spécifique
$ id                                  # UID, GID et tous les groupes
```

3.4.3 Le umask

Quand vous créez un nouveau fichier ou répertoire, Linux leur attribue des permissions par défaut. Ces permissions ne sont pas arbitraires : elles résultent du umask (user file creation mask), un masque qui soustrait des permissions des valeurs maximales. Les valeurs maximales sont 666 pour les fichiers (jamais de x par défaut pour des raisons de sécurité) et 777 pour les répertoires. Le umask par défaut sous Ubuntu est 0022, ce qui donne $666-022=644$ pour les fichiers et $777-022=755$ pour les répertoires. Comprendre le umask est important pour les scripts et les déploiements automatisés car il affecte les permissions de tous les fichiers créés.

```
$ Terminal

# Afficher le umask actuel
$ umask
# 0022

# Calcul des permissions par défaut
# Fichier max : 666 - umask 022 = 644 (rw-r--r--)
# Dossier max : 777 - umask 022 = 755 (rwxr-xr-x)
```

```
# Modifier temporairement le umask (valable pour la session courante)
$ umask 027          # Nouveau umask : fichiers=640, dossiers=750
$ touch test.txt && ls -l test.txt
# -rw-r----- : permissions 640 appliquées

# Modifier le umask de façon permanente
$ echo "umask 027" >> ~/.bashrc  # Ajoute dans le profil utilisateur

# Exemples de umask courants
# umask 022 → défaut Ubuntu : fichiers 644, dossiers 755
# umask 027 → plus restrictif : fichiers 640, dossiers 750
# umask 077 → très restrictif : fichiers 600, dossiers 700 (tout privé)
```

3.5 Permissions Spéciales : SUID, SGID et Sticky Bit

3.5.1 Le SUID

Le bit SUID (Set User ID) est une permission spéciale qui s'applique aux fichiers exécutables. Quand un programme avec le bit SUID est exécuté, il s'exécute avec les privilèges du propriétaire du fichier, et non avec les privilèges de l'utilisateur qui le lance. L'exemple classique est la commande `passwd` : elle appartient à root et possède le bit SUID, ce qui permet à un utilisateur normal de modifier son propre mot de passe malgré que `/etc/shadow` appartient à root. Sans SUID, `passwd` nécessiterait les droits root. Le SUID est signalé par un `s` à la place du `x` dans les permissions du propriétaire. Si le fichier n'était pas exécutable, le `s` apparaît en majuscule `S` (erreur de configuration).

```
$ Terminal

# Identifier un fichier avec SUID
$ ls -l /usr/bin/passwd
# -rwsr-xr-x 1 root root 68208 mars 14 2023 /usr/bin/passwd
#      ^
#      └ "s" à la place de "x" : SUID activé

# Trouver tous les fichiers SUID sur le système
$ find / -perm -4000 -type f 2>/dev/null

# Activer le SUID sur un fichier
$ sudo chmod u+s mon_programme      # Notation symbolique
$ sudo chmod 4755 mon_programme     # Notation octale (4xxx)
```

```
# Désactiver le SUID
$ sudo chmod u-s mon_programme

# ⚠ Attention sécurité : le SUID mal utilisé est une faille critique
# Un script shell avec SUID peut être exploité pour escalader les privilèges
```

3.5.2 Le SGID

Le bit SGID (Set Group ID) fonctionne de façon similaire au SUID mais au niveau du groupe. Sur un fichier exécutable, SGID fait tourner le programme avec les droits du groupe propriétaire. Sur un répertoire, SGID a un comportement différent et très utile pour la collaboration : tout fichier créé dans ce répertoire hérite automatiquement du groupe du répertoire, et non du groupe primaire de l'utilisateur qui crée le fichier. C'est une fonctionnalité précieuse pour les répertoires partagés d'équipe : tous les fichiers créés appartiennent automatiquement au bon groupe, sans que chaque développeur n'ait à faire `chgrp` manuellement.

```
$ Terminal

# SGID sur un RÉPERTOIRE (cas le plus utile)
$ sudo mkdir /shared/projet-equipe
$ sudo chown :devs /shared/projet-equipe
$ sudo chmod g+s /shared/projet-equipe    # Active SGID sur le répertoire
$ ls -ld /shared/projet-equipe
# drwxrwsr-x 2 root devs 4096 mars 4 10:00 /shared/projet-equipe
#      ^
#      └ "s" sur le groupe : SGID activé

# Maintenant, tout fichier créé dans ce dossier appartient au groupe "devs"
$ touch /shared/projet-equipe/code.py
$ ls -l /shared/projet-equipe/code.py
# -rw-rw-r-- 1 alice devs 0 mars 4 10:05 code.py
#      ^^^^
#      Automatiquement hérité du répertoire parent !

# Activer SGID en octal : chmod 2xxx
$ sudo chmod 2775 /shared/projet-equipe    # 2=SGID, 775=rwxrwxr-x
```

3.5.3 Le Sticky Bit

Le Sticky Bit est une permission spéciale appliquée aux répertoires partagés. Quand un répertoire possède le Sticky Bit, seul le propriétaire d'un fichier (ou root) peut supprimer ou renommer ce fichier, même si d'autres utilisateurs ont les droits d'écriture dans le répertoire. L'exemple canonique est /tmp : ce répertoire est accessible en écriture par tous les utilisateurs (permissions 1777), mais sans Sticky Bit, n'importe qui pourrait supprimer les fichiers temporaires des autres. Grâce au Sticky Bit, Alice ne peut supprimer que ses propres fichiers dans /tmp, et Bob ne peut supprimer que les siens. Le Sticky Bit est signalé par un t à la place du x dans les permissions des autres utilisateurs.

```
$ Terminal

# Vérifier le Sticky Bit sur /tmp
$ ls -ld /tmp
# drwxrwxrwt 15 root root 4096 mars 4 12:00 /tmp
#
#      ^
#      |
#      └─ "t" : Sticky Bit activé

# Activer le Sticky Bit
$ sudo chmod +t /dossier/partage
$ sudo chmod 1777 /dossier/partage      # 1xxx en octal

# Désactiver le Sticky Bit
$ sudo chmod -t /dossier/partage

# Résumé des permissions spéciales en octal
# chmod 4xxx : SUID   (ex: 4755)
# chmod 2xxx : SGID   (ex: 2775)
# chmod 1xxx : Sticky Bit (ex: 1777)
# chmod 6xxx : SUID + SGID combinés
```

Permission spéciale	Sur un fichier	Sur un répertoire	Commande
SUID (4xxx)	Exécute avec les droits du propriétaire	Pas d'effet standard	chmod u+s ou chmod 4xxx
SGID (2xxx)	Exécute avec les	Nouveaux fichiers	chmod g+s ou chmod

Permission spéciale	Sur un fichier	Sur un répertoire	Commande
	droits du groupe	héritent du groupe	2xxx
Sticky Bit (1xxx)	Ignoré sur les fichiers modernes	Seul le propriétaire peut supprimer son fichier	chmod +t ou chmod 1xxx

TRAVAIL PRATIQUE 3 : Manipulation des Fichiers et Gestion des Permissions

Objectifs du TP 3

- Manipuler des fichiers et répertoires avec cp, mv, rm
- Lire et interpréter les permissions affichées par ls -l
- Modifier les permissions avec chmod en notations symbolique et octale
- Changer propriétaire et groupe avec chown et chgrp
- Mettre en pratique les bonnes pratiques de sécurité des fichiers

Exercice 3.1 : Manipulation de Fichiers et Répertoires (25 min)

Exécutez les commandes suivantes dans l'ordre et observez les résultats :

```
$ Terminal

# Étape 1 : Créer l'arborescence de travail
$ mkdir -p ~/TP3-Linux/{projets/{web,scripts,data},archives,temp}
$ tree ~/TP3-Linux          # Visualiser la structure créée

# Étape 2 : Créer des fichiers de test
$ echo "Script principal" > ~/TP3-Linux/projets/scripts/main.sh
$ echo "Page HTML" > ~/TP3-Linux/projets/web/index.html
$ echo "Base de données" > ~/TP3-Linux/projets/data/db.sqlite
$ touch ~/TP3-Linux/temp/{cache1.tmp,cache2.tmp,cache3.tmp}

# Étape 3 : Copier des fichiers
$ cp ~/TP3-Linux/projets/scripts/main.sh ~/TP3-Linux/archives/main.sh.bak
$ cp -r ~/TP3-Linux/projets/web ~/TP3-Linux/archives/web-backup
$ ls ~/TP3-Linux/archives/

# Étape 4 : Déplacer et renommer
$ mv ~/TP3-Linux/projets/data/db.sqlite ~/TP3-Linux/projets/data/prod.db
```

```
$ mv ~/TP3-Linux/temp ~/TP3-Linux/projets/temp
$ ls ~/TP3-Linux/projets/

# Étape 5 : Supprimer proprement
$ rm ~/TP3-Linux/projets/temp/*.tmp
$ ls ~/TP3-Linux/projets/temp/
$ rmdir ~/TP3-Linux/projets/temp/
$ tree ~/TP3-Linux
```

Exercice 3.2 : Lecture des Permissions (20 min)

Pour chaque ligne de `ls -l` ci-dessous, décomposez et expliquez les permissions :

Sortie <code>ls -l</code>	Type	Perms User	Perms Group	Perms Others	Que peut faire chaque classe ?
<code>-rwxr-xr-x root root /bin/ls</code>	Fichier	rwx	r-x	r-x	à compléter par l'étudiant
<code>-rw----- etudiant etudiant .bashrc</code>	Fichier	rw-	---	---	à compléter par l'étudiant
<code>drwxr-xr-x root root /etc</code>	Répertoire	rwx	r-x	r-x	à compléter par l'étudiant
<code>-rwsr-xr-x root root /usr/bin/passwd</code>	Fichier SUID	rws	r-x	r-x	à compléter par l'étudiant
<code>drwxrwxrwt root root /tmp</code>	Répertoire	rwx	rwx	rwt	à compléter par l'étudiant

Exercice 3.3 : Modifier les Permissions (30 min)

Créez les fichiers suivants et appliquez les permissions indiquées avec `chmod` :

```
$ Terminal

# Créer les fichiers de test
$ cd ~/TP3-Linux
$ touch script_public.sh cle_privee.pem config_equipe.conf page_web.html

# ---- Tâche 1 : script_public.sh ----
# Objectif : exécutable par tous, modifiable seulement par le propriétaire
# Permissions cibles : rwxr-xr-x (755)
```

```
$ chmod 755 script_public.sh
$ ls -l script_public.sh # Vérifiez : -rwxr-xr-x

# ---- Tâche 2 : cle_privee.pem ----
# Objectif : accessible UNIQUEMENT par le propriétaire
# Permissions cibles : rw----- (600)
$ chmod 600 cle_privee.pem
$ ls -l cle_privee.pem # Vérifiez : -rw-----

# ---- Tâche 3 : config_equipe.conf ----
# Objectif : propriétaire lecture+écriture, groupe lecture seule, autres rien
# Permissions cibles : rw-r----- (640)
$ chmod 640 config_equipe.conf
$ ls -l config_equipe.conf # Vérifiez : -rw-r-----

# ---- Tâche 4 : page_web.html ----
# Utiliser la notation SYMBOLIQUE
# Objectif : lecture pour tous, écriture seulement pour propriétaire
$ chmod u=rw,g=r,o=r page_web.html
$ ls -l page_web.html # Vérifiez : -rw-r--r--

# ---- Tâche 5 : notation symbolique avancée ----
# Ajouter exécution pour tout le monde sur script_public.sh
$ chmod a+x script_public.sh
# Retirer écriture du groupe sur config_equipe.conf
$ chmod g-w config_equipe.conf
```

Exercice 3.4 : Propriété et Permissions Avancées (25 min)

```
$ Terminal

# Étape 1 : Observer la propriété actuelle
$ ls -la ~/TP3-Linux/

# Étape 2 : Créer un répertoire partagé (simuler du travail collaboratif)
$ mkdir ~/TP3-Linux/partage
$ ls -ld ~/TP3-Linux/partage

# Étape 3 : Configurer le SGID pour héritage de groupe
$ sudo chown :etudiant ~/TP3-Linux/partage # Assigne le groupe
$ sudo chmod 2775 ~/TP3-Linux/partage # SGID + rwxrwxr-x
$ ls -ld ~/TP3-Linux/partage
```

```
# Cherchez le "s" sur la position du groupe : drwxrwsr-x

# Étape 4 : Tester l'héritage SGID
$ touch ~/TP3-Linux/partage/test_sgid.txt
$ ls -l ~/TP3-Linux/partage/

# Le groupe du fichier créé doit être "etudiant" (hérité du répertoire)

# Étape 5 : Sticky Bit sur /tmp
$ ls -ld /tmp          # Observez le "t" dans les permissions
$ touch /tmp/mon_fichier_temp.txt
$ ls -l /tmp/mon_fichier_temp.txt

# Étape 6 : Trouver les fichiers SUID sur le système
$ find /usr/bin -perm -4000 -type f 2>/dev/null

# Notez les programmes trouvés et réfléchissez : pourquoi ont-ils besoin du
SUID ?

# Étape 7 : umask
$ umask                # Affichez le umask actuel
$ umask 077            # Changez le umask (très restrictif)
$ touch test_umask.txt && ls -l test_umask.txt

# Les permissions doivent être 600 (rw-----)
$ umask 022            # Restaurez le umask par défaut
```